QCon
全球软件开发大会

InfoQ 技术大会

快手智能运维助手实践

吴垚

高级技术专家



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 快手为什么做智能运维助手

02 运维助手的形态与能力

03 运维助手的进化

04 业务落地实践

05 总结与展望

06

01

为什么做智能运维助手

工具都齐了，为什么运维还是很累

研发和运维工作分布



研发侧工作　　运维侧工作

人工串联系统

各类平台能力已经存在，但需要人手在系统之间切换和串联



工具齐全，但工具之间不会自动组成事件处理链路

时间碎片化

工作被会议、IM、告警、审批和临时排障不断打断，难以专注



频繁的上下文切换，让研发和运维难以保持专注
降低了问题处理效率

从DevOps到AIOps再到AgenticOps

AIOps的困境：发现异常不代表处理事件

行业演进重点，也在从“把工具建起来、实现智能分析”，**走向“把事件处理链路串起来”、实现运维闭环**

	2009年 DevOps阶段	2016年 AIOps阶段	≈2025年 AgenticOps（当前）
解决的问题	开发与运维割裂 交付效率低 流程不规范	异常难发现 告警多 定位效率低	跨系统信息查询成本高 人工串联流程重 复杂场景协同效率低
侧重点	工具链打通 流程标准化 CI/CD 研发运维协同	异常检测 告警降噪 根因定位	事件理解 上下文补齐 工具调用 处置建议
局限性 / 阶段特征	工具虽然打通 问题处理仍靠人	偏单点能力强化 尚未形成完整处理闭环	系统先处理 人做关键节点确认 事件驱动闭环

大模型和Agent让运维有了新可能

运维的本质：事件 + 数据 + 推理 + 行动

事件

- 告警
- 变更
- 巡检
- 应急

这些运维事件已经在
线上系统中产生和流转。

数据

- 指标
- 日志
- Trace
- 变更详情
- 服务信息
- 链路拓扑

运维数据持续沉淀
在各类系统中。

推理

- 上下文理解
- 信息归纳
- 初步判断

大模型让分散信息有机会
被串起来，辅助判断问题
是否真实、影响范围和可能原因。

行动

- 调用工具
- 编排流程
- 持续执行任务

Agent 让分析结果有机会
进入后续处理流程。

过去真正依赖人的，是推理和行动；现在，运维事件开始有机会先由助手接住，再由人确认。

我们想把快手运维Agent做成什么样



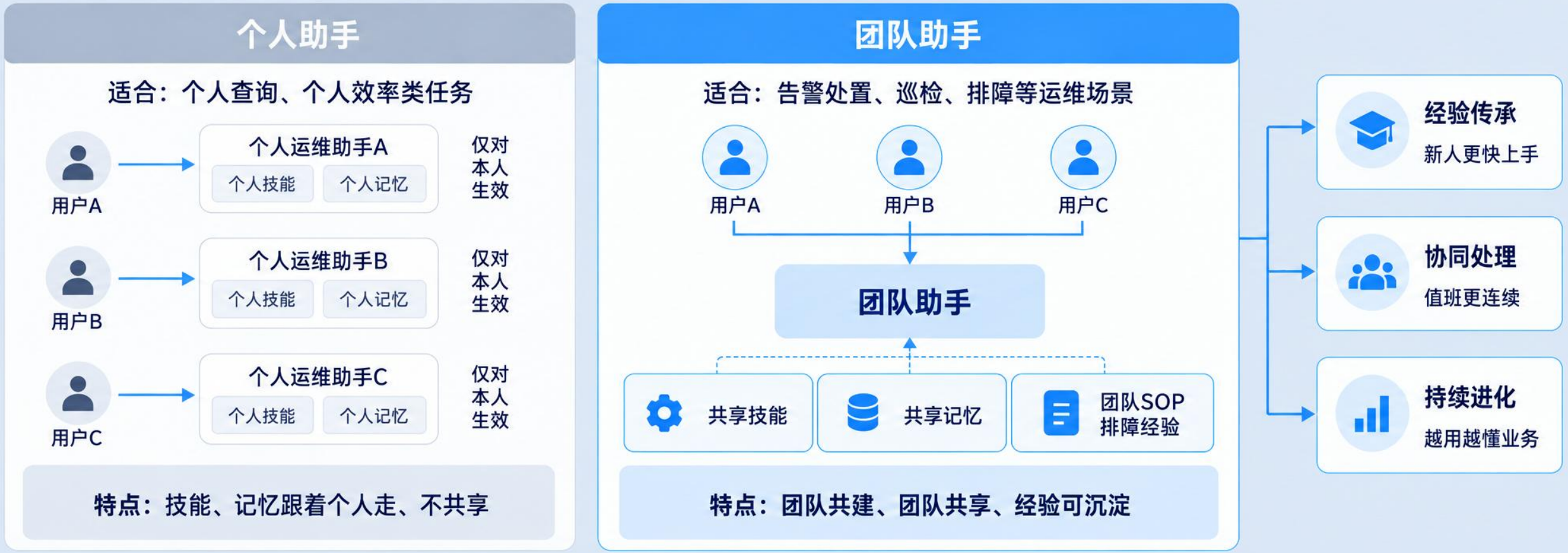
让事件进入助手，由助手驱动处理流程

人从基础查询与信息拼接中解放出来，专注确认、决策和兜底

02

运维助手的形态与能力

产品形态：团队助手 & 个人助手



运维助手—事件触发能力

支持范围

触发事件

- 告警
- 变更
- 应急
- 定时巡检

对话是可随时发起的兜底入口

告警支持平台

- **业务告警**：业务指标与核心链路
- **服务告警**：RPC、HTTP、CPU、内存、JVM、异常堆栈
- **数据库告警**：关系型、缓存、搜索、固存储
- **客户端告警**：端上埋点与体验指标
- **中间件与大数据**：端息队列、流计算
- **物理机告警**：宿主机层指标

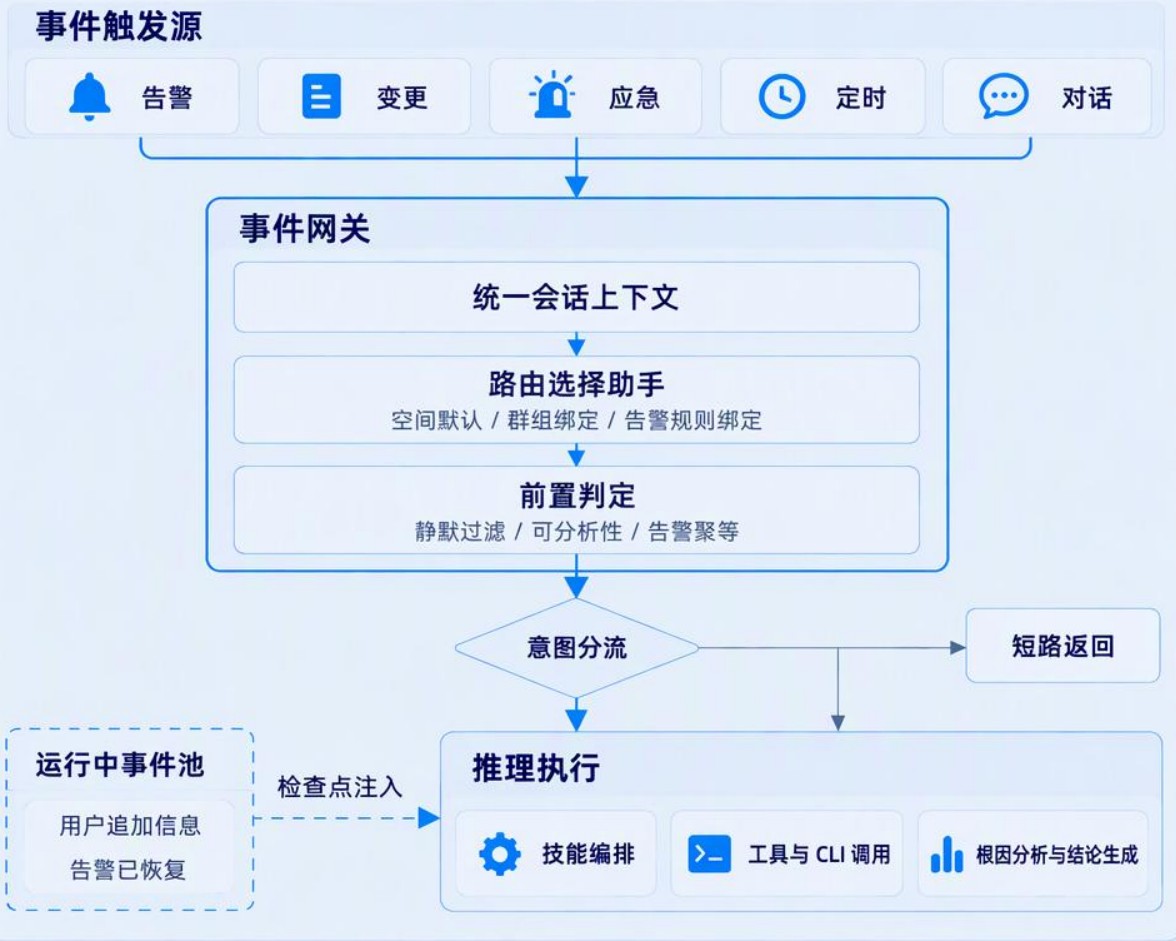
变更支持平台

- 容器部署
- 配置变更
- AB 实验
- 功能开关
- 数据库变更
- 服务治理配置

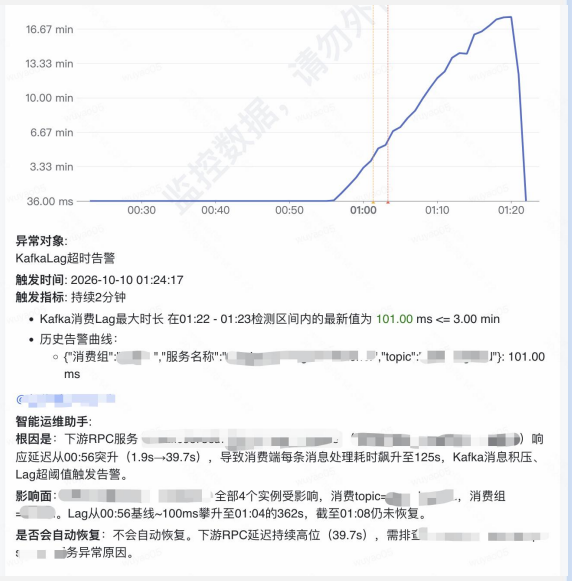
结果回传

- **告警**：结论输出在告警消息思
- **变更**：结论发送到指定群或人
- **应急**：分析结果推送到应急群
- **定时**：执行结果按配置推送

技术架构



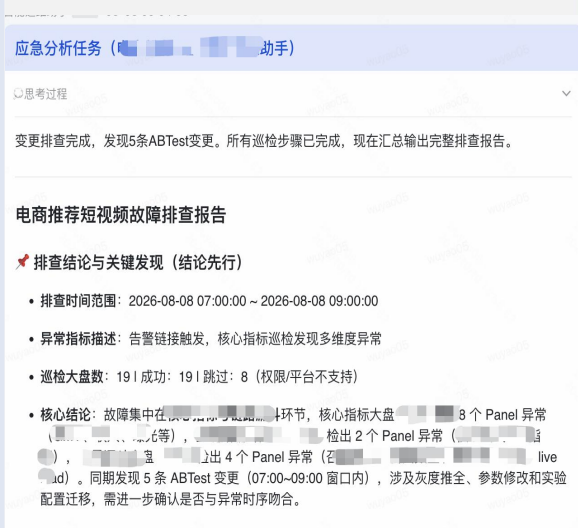
运维助手—事件触发效果



告警触发



变更触发



应急触发



定时触发

QCon 全球软件开发大会
InfoQ 技术大会

运维助手—运维技能生态

全量技能分类

观测诊断
从指标、日志到链路快速定位问题

指标与大盘查询	日志检索与聚合	链路与慢调用分析	告警查询与解释
指标查询	日志查询	调用链查询	告警查询
大盘巡检	错误聚合	慢调用分析	告警解释

变更与部署
变更可追溯，发布更安全

变更查询与关联	部署诊断与实例操作	配置与功能开关
变更查询	实例诊断	配置查询
变更关联	实例操作	功能开关管理

服务与代码
掌握服务状态，快速定位代码问题

服务信息与拓扑		代码检索与堆栈定位	
服务查询	服务拓扑	代码检索	堆栈定位

数据与中间件
洞察数据与中间件健康

缓存查询与管理	数据库慢查询与容量
缓存命中与容量	慢查询与连接数

行动与治理
从应急到治理，保障稳定运行

应急协同	定时巡检	告警治理

技术架构

技能生态

内置运维技能	通用查询技能	处置技能	助手管控技能	业务自定义技能
根因定位	指标查询	实例重启	定时任务	800+
大盘巡检	日志查询	服务升级	技能管理	覆盖各业务线的自定义运维技能
变更分析	变更查询	告警静默	知识库管理	
告警治理	服务查询	应急通告		
部署诊断	代码查询			

技能决定在什么场景、按什么顺序调用哪些 CLI。

统一 CLI

统一入口 | 鉴权与凭证 | 参数归一 | 结果结构化

把各平台能力收敛为可被助手直接调用的工具。

运维平台层

监控指标	告警	日志	Trace	变更
服务信息	代码仓库	容器部署	数据库	配置中心
功能开关	AB 实验	oncall 平台	devops 平台	

运维助手—运行管控能力

管控能力

策略可配置

- 管控策略由人评估影响面后配置
- 策略决定动作的风险等级与管控粒度

执行前拦截

- 每次执行技能前先经过管控策略引擎
- 技能不会绕过管控直接执行

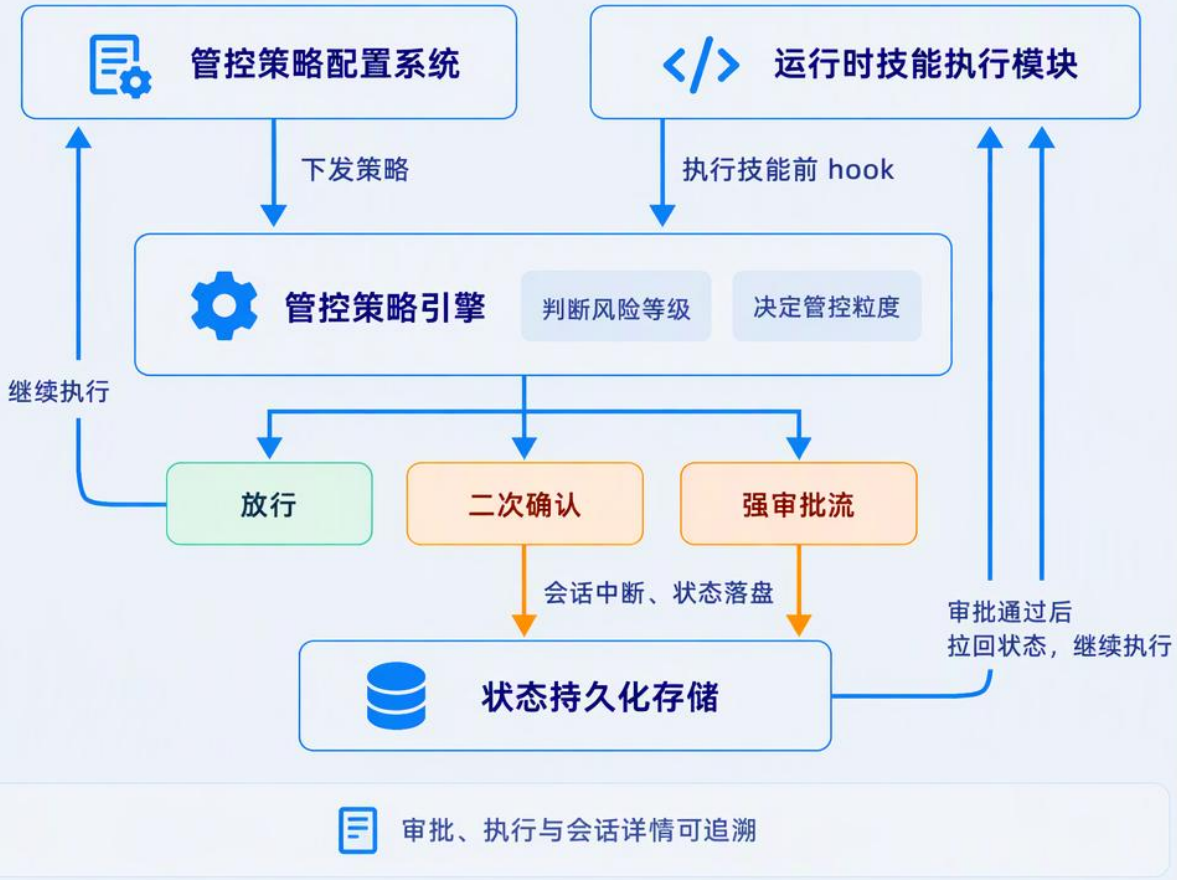
三档管控粒度

放行	低风险直接继续执行
二次确认	执行前需人工确认
强审批流	进入完整审批流程

会话可恢复

- 二次确认与强审批流会中断当前会话
- 会话状态持久化保存
- 审批通过后从持久存储拉回状态，继续执行

技术架构



03

运维助手的进化

定位不准，怎么解决？

助手看到了什么

已有能力	通用知识 + 平台数据都能拿到
现象	核心接口错误率升高
证据	异常发生前有一批变更发布
证据	日志中出现少量超时
推断	变更最可能是根因
建议	优先回滚该变更

人工复核的结论

- 该变更与异常链路无关
- 少量超时在高峰期属已知基线
- **真实根因是下游容量不足**
- 扩容下游资源后异常恢复

通用知识和平台数据都不缺，缺的是领域知识

同一个服务、同一类告警，这已经不是第一次判错

这类问题为什么总在犯？

通用知识、记忆在运维场景的问题

文字相似，不等于应该召回

出问题的对象	同样是超时，不同服务、不同实例，根因完全不同
现象	报错文案相似，但报错层级和形态不一样，指向的组件就不同
时间	同一个现象发生在发布后、高峰期、还是深夜，优先级不一样
条件	环境、版本、拓扑位置、流量规模不同，同一结论未必成立

只对上其中一两条文字，就能凑出一段看起来很合理的结论

错方向和没方向

给错根因	方向被带偏，排障时间被浪费
	结论看起来合理，没人会怀疑
	错的结论还会写回记忆，下次继续错

后者的代价明显更低

没给根因	助手承认证据不足
	人接管，损失可控
	不会污染已有知识

给错根因，比不给根因更坑人

运维领域知识有哪些

领域知识分三类

01

记什么
服务与指标特性

- 服务的核心指标与正常波动范围
- 指标之间的业务关系
- 服务特有的异常判断条件
- 已确认的异常模式

→ 记在哪
记忆 Memory

02

记什么
系统结构与排障方法

- 排障 SOP 与处置顺序
- 服务架构与上下游关系
- 代码结构与调用关系
- 配置与依赖关系

→ 记在哪
知识库 Knowledge Base

03

记什么
处置规律与根因分布

- 告警规则与根因类型的对应
- 根因概率分布
- 排障优先级
- 经验的适用范围与置信度

→ 记在哪
经验 Experience

运维领域知识怎么召回

三类知识怎么召回

记忆 Memory

召回方式

三个维度精确匹配：服务、环境、时间窗

补充

BM25 关键词、向量语义、精确维度匹配三路并行，再合并过滤

约束

不做无边界的语义扩散

知识库 Knowledge Base

召回方式

语义向量、知识图谱遍历、BM25 关键词三路混合

补充

助手自己决定检索什么、从哪检索，不充分就继续下钻

约束

文本相似只是其中一路，图谱负责沿依赖走

经验 Experience

召回方式

条件联合匹配

补充

症状相似 + 拓扑位置相近 + 时间环境吻合 + 置信度过门槛

约束

不能只按文本相似

召回的难点不是找得多，是每条知识都要对得上对象、现象、时间、条件

运维助手评测实践

评测要记录什么

- 实际召回了哪些内容
- 哪些内容最终进入了上下文
- 调用了哪些技能与工具
- 关键执行路径是否真的执行
- 每一步的中间判断
- 最终答案引用了哪些证据

只评最终答案，就分不清是哪一层错了

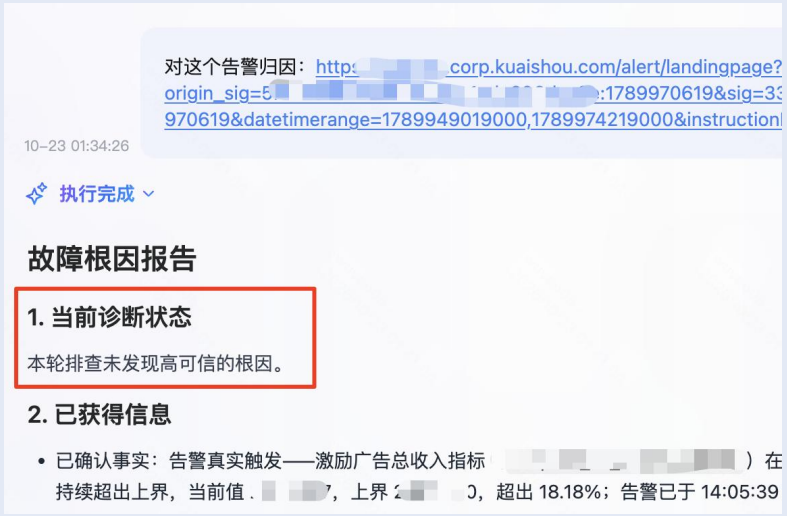
失败之后往哪调

评测发现	典型证据	该调哪里
不知道服务特有的异常条件	召回里没有指标基线	**记忆 Memory**
不知道关键上下游	知识检索没返回架构关系	**知识库**
排障顺序不符历史分布	低概率根因被优先排查	**经验 Experience**
依赖关系召回了却没被用上	检索到架构信息，结论仍未沿依赖下钻 前者是知识库缺内容，后者是内容召回不上来	**知识库**
路径对但工具失败	参数或执行结果异常	**工具链**

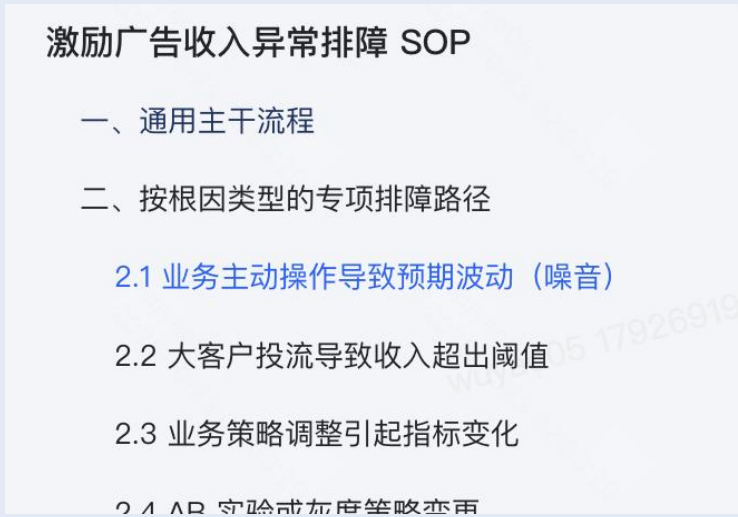
评测的价值不是给分，是告诉系统该改哪一层

从不准到准：一个进化的Case

业务问题，助手给不出根因



总结SOP，录入知识库



评测，迭代SOP



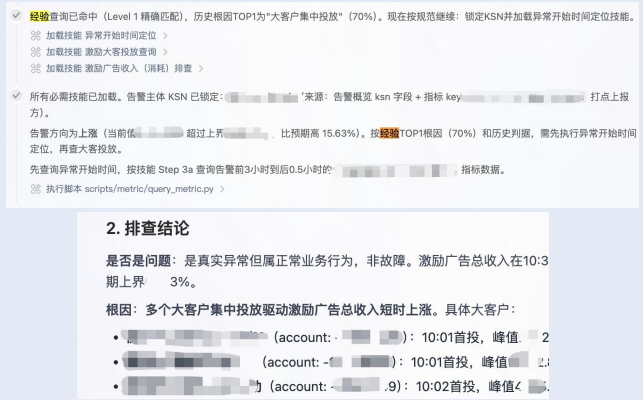
用户持续打标反馈



真实根因分布沉淀为经验



召回经验、知识，实现进化



持续迭代


领域知识
（记忆 · 知识库 · 经验）
记忆
• 服务特性
• 指标特性
• 异常判断条件
知识库
• 运维 SOP
• 服务架构
• 代码结构
经验
• 根因类型
• 根因概率分布
• 排障优先级
检索使用
更新沉淀
1
召回
记忆、知识库、经验
进入分析上下文
2
处置
• 助手分析事件并给出处置建议
• 人做关键确认
3
反馈
收集实际根因、是否采纳、
结果是否有效
4
评测
• 从结果准确性、结果稳定性、
关键路径达成率、时延、Token 消耗
进行综合评估
• 验证效果并识别退化
5
回流校准
更新记忆、知识和经验
6
下次更准
相似事件优先使用
已校准的领域知识
每次处置都在沉淀
沉淀的东西能被召回，
召回效果通过评测持续校准

运维助手进化的几个坑

什么都往记忆里写

噪声膨胀，召回不稳

只存结论化、主体绑定、可复用的

知识库当成网盘用

能搜到文档，支撑不了推理

保留章节、服务与依赖结构

把最近变更当成强先验

时间相关性被当成因果

必须校验影响实例与异常链路

经验只记成功

置信度虚高，错误经验不衰减

采纳、否定、无用、最终根因全记

召回越多越好

上下文污染，被次要证据带偏

设检索预算、阀值与充分性判断

只评最终答案

分不清召回错、推理错还是工具错

同时评结果与关键执行路径

问题大多不在模型，在知识边界没画清

03

运维助手的业务实践

运维助手的业务场景

业务场景全景

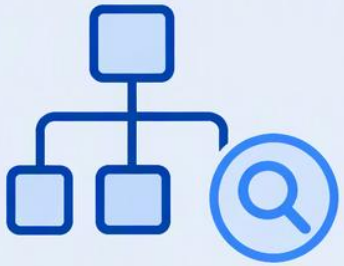
多类场景共享“事件触发→自动分析→结果推送→人工决策或确认”的基本模式。



大盘巡检



告警降噪识别、归因



复杂链路排障



简单问题处置



总结问题

事件触发 → 自动分析 → 结果推送 → 人工决策/确认

巡检：从人工盯盘到问题的自动发现、升级、总结

业务问题

核心业务大盘依赖人工定时巡检，发现问题后还要拉人定位、催进展、写总结。

- 巡检频次有限，夜间与节假日存在盲区
- 人力投入大，工作被频繁打断
- 执行标准不统一，问题记录易遗漏
- 判断经验留在个人脑中，换人容易断档

助手怎么用

1 前置配置

配置业务巡检规则
同环比、固定阈值等判断标准

自定义总结技能
定义日报结构并组合查询能力

配置定时巡检任务
挂载规则、总结技能与推送渠道

2 日常运转

问题发现
定时巡检，命中规则后推送，人做确认

→

问题跟进
负责人接手，超时再次提醒并升级

→

问题总结
自动汇总问题，关联告警与应急信息，形成日报

效果与模式变化

之前	之后
人工轮值，固定时间看大盘	助手 7×24 自动巡检
人工在群里发出问题	助手自动发出问题，人做确认
值班长手写问题总结	助手自动写总结，人做信息补充
判断经验跟着个人走	判断经验沉淀为团队知识

从“人工盯盘”转为“助手发现与跟进，人做关键确认”。

告警噪音识别与归因

业务问题

核心交易系统稳定性要求高，
需要研发同学轮值处理告警。

- 每条告警先由人工判断是不是噪音
- 非噪音告警还要继续定位根因、解决问题
- 需要跨平台翻查指标、日志、链路和变更数据
- 值班周期大量精力耗在告警处理，难以投入研发

人的注意力被大量消耗在重复排查上。

助手用法

值班同学不再逐个平台翻数据，只看结论做确认。



应用效果

- 一部分噪音告警可略过人工排查
- 归因准确时，人工介入显著减少
- 值班同学更快看清问题与根因
- 人的精力回到异常确认和问题解决

运维模式变化

之前	之后
人工逐条看告警	助手先做降噪初判
人工先判断噪音	助手自动汇总数据并归因
人工跨平台查数据	助手推送带结论的消息
人工整理结论	人员看真问题并做确认

从“逐条排查”转为“先看结论”。

复杂链路排障

业务问题

复杂问题拉起应急后，要逐个召集上下游并搜集问题现象，人工汇总后才能分析影响范围。

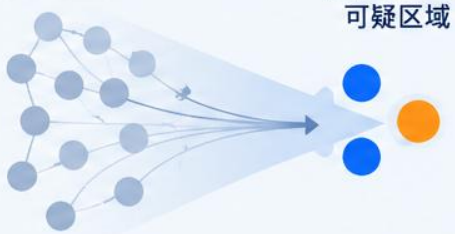
- 上下游系统多，现象分散
- 人工逐个搜集和对齐信息
- 汇总后才能做初步范围分析
- 大量时间用在凑信息，而不是决策和止损

助手用法

1. **应急事件触发** → 配置应急任务后，应急事件自动触发助手巡检。
2. **自动巡检** → 复用大盘巡检中已经定义的业务知识与判断规则。
3. **汇总上下游观测数据** → 对上下游链路与观测数据进行汇总分析。
4. **识别异常**
5. **初步判断范围**
6. **生成自检报告** → 巡检结论按需组装为自检报告。

应用效果

- 异常现象自动收集和汇总
- 初步范围更早收敛
- 自检报告可直接用于定位与复盘
- 人从信息搜集转向根因判断与处置



运维模式变化

之前	之后
人工拉上下游进群	应急自动触发助手
人工逐个搜集现象	助手汇总观测数据
人工汇总数据	助手先摆出异常现象和可疑范围
汇总后才开始范围分析	人聚焦决策、止损和根因定位

助手把现象和范围先摆到桌面上，人的时间用在决策和止损。

简单问题处置

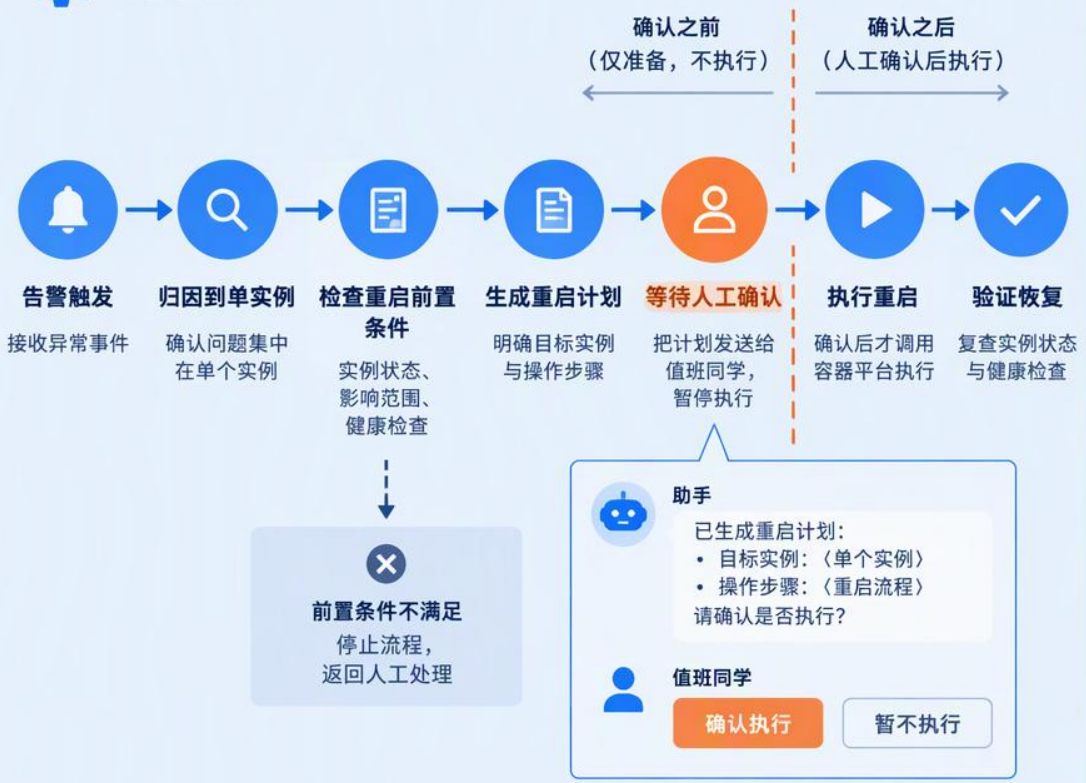
业务问题

单实例异常通常问题明确、处置路径固定，但人工仍需跨系统确认状态并执行重启。

- 告警到达后先人工判断是否为单实例问题
- 人工检查实例状态、影响范围和重启条件
- 人工编排重启步骤并等待合适窗口
- 重启后还要人工验证服务是否恢复

简单问题流程固定，却持续打断值班同学。

助手用法



应用效果

- 自动完成问题初判与前置检查
- 重启计划结构化、可核验
- 确认后执行，降低误操作风险
- 重启后自动验证恢复状态

人做关键确认，助手执行固定流程。

运维模式变化

之前	之后
人工判断异常实例	助手识别单实例问题
人工查询多项状态	助手检查前置条件并生成计划
人工编排并执行重启	人工确认后助手执行重启
人工回看是否恢复	助手验证结果并回传

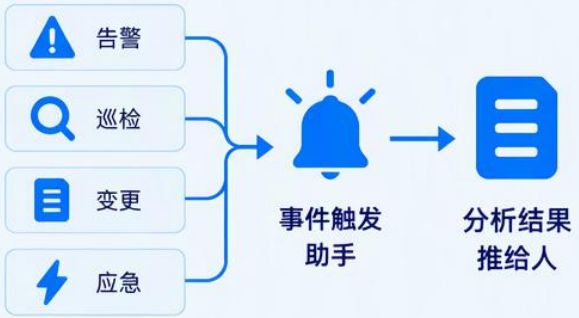
从“人工跑完整流程”转为“助手准备并执行，人做关键确认”。

05

总结与展望

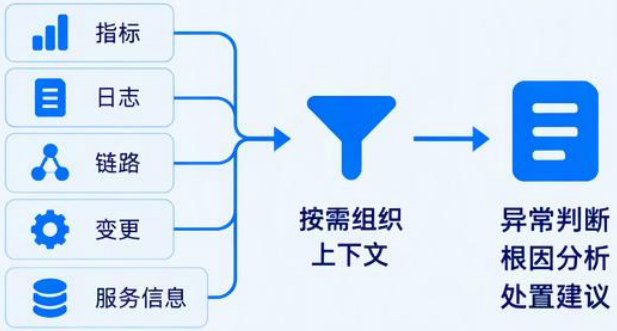
经验总结

1 事件先于对话



- 运维工作天然由告警、巡检、变更、应急等事件驱动
- 让事件自动触发助手，比等待用户发起提问更适合运维场景
- 助手先接住事件，再把分析结果推给人

2 上下文决定判断质量



- 先补齐指标、日志、链路、变更和服务信息
- 再进行异常判断、根因分析和处置建议
- 数据不是简单堆给模型，而是围绕当前事件按需组织

3 领域知识决定能力上限



- 记忆承载服务特性、指标特性和异常判断条件
- 知识库承载运维 SOP、服务架构和代码结构
- 经验承载根因类型、根因概率分布和排障优先级
- 沉淀的知识必须能在新事件中被召回

4 高风险动作必须受控



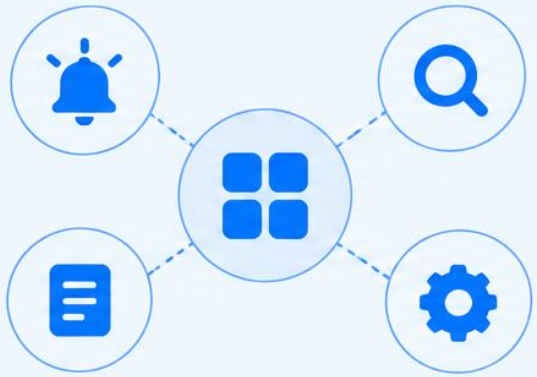
- 助手负责准备、分析和执行固定流程
- 高风险动作由人确认后再执行
- 执行完成后验证结果，不把“发起动作”当成“问题解决”

通用模型提供推理能力，运维数据与团队经验决定助手是否真正可用

未来展望

01 覆盖更多场景

对更多的运维工作提效



从单一场景向告警、巡检、
变更、应急等更多运维工作扩展。

02 增强处置能力

在风险可控的基础上，增强更多
类型问题的处置能力



能力增强必须经过风险控制与人工确认。

03 扩大进化范围

从 RCA 场景的进化，扩展到
通用场景的进化



让问题先找助手，让人专注决策和止损

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software

